

第10章 对文件的输入输出

10.1 C文件的有关基本知识

10.1.1 什么是文件

10.1.2 文件名

10.1.3 文件的分类

10.1.4 文件缓冲区

10.1.5 文件类型指针

10.1.1 什么是文件

- “文件”指存储在外部介质上数据的集合
 - 一批数据是以文件的形式存放在外部介质上的
 - 操作系统是以文件为单位对数据进行管理
 - 想找存放在外部介质上的数据，先按文件名找到所指定的文件，然后再从该文件读数据
 - 要向外部介质上存储数据也必须先建立一个文件（以文件名作为标志），才能向它输出数据

10.1.1 什么是文件

- 文件有不同的类型，在程序设计中，主要用到两种文件：

- (1) **程序文件**。包括**源程序文件**(后缀为.c)、**目标文件**(后缀为.obj)、**可执行文件**(后缀为.exe)等。这种文件的内容是程序代码。
- (2) **数据文件**。

10.1.1 什么是文件

- 文件有不同的类型，在程序设计中，主要用到两种文件：
 - (1) 程序文件。
 - (2) 数据文件。文件的内容不是程序，而是供程序运行时读写的数据，如在程序运行过程中输出到磁盘(或其他外部设备)的数据，或在程序运行过程中供读入的数据。如一批学生的成绩数据，或货物交易的数据等。
- 本章主要讨论的是数据文件

10.1.1 什么是文件

- 在以前各章中所处理的数据的输入和输出，从终端的键盘输入数据，运行结果输出到终端显示器上
- 实际应用中，常常需要将一些数据输出到磁盘上保存起来以便以后使用，这就要用到磁盘文件

10.1.1 什么是文件

- 操作系统把各种设备都统一作为文件处理
- 从操作系统的角度看，每一个与主机相联的输入输出设备都看作是文件。例如，
 - 终端键盘是输入文件
 - 显示屏和打印机是输出文件

10.1.1 什么是文件

- 输入输出是数据传送的过程，数据如流水一样从一处流向另一处，因此常将输入输出形象地称为流(**stream**)，即数据流。流表示了信息从源到目的端的流动。
- 输入操作时，数据从文件流向计算机内存
- 输出操作时，数据从计算机流向文件
- 无论是用Word打开或保存文件，还是C程序中的输入输出都是通过操作系统进行的

10.1.1 什么是文件

- “流”是一个传输通道，数据可以从运行环境流入程序中，或从程序流至运行环境
- 从C程序的观点来看，无论程序一次读写一个字符，或一行文字，或一个指定的数据区，作为输入输出的各种文件或设备都是统一以逻辑数据流的方式出现的。C语言把文件看作是一个字符（或字节）的序列。一个输入输出流就是一个字符流或字节(内容为二进制数据)流。

10.1.1 什么是文件

- C 的数据文件由一连串的字符（或字节）组成，而不考虑行的界限，两行数据间不会自动加分隔符，对文件的存取是以字符（字节）为单位的。输入输出数据流的开始和结束仅受程序控制而不受物理符号（如回车换行符）控制，这就增加了处理的灵活性。这种文件称为流式文件。

10.1.2 文件名

- 文件要有一个唯一的文件标识，以便用户识别和引用。
- 文件标识包括三部分：
 - (1) 文件路径
 - (2) 文件名主干
 - (3) 文件后缀

10.1.2 文件名

文件路径

表示

文件名主干

文件后缀

置。如：

D: \CC\temp\file1.dat

- 表示file1.dat文件存放在D盘的CC目录下的

命名规则遵循标识符
的命名规则

一般不超过3个字母
(doc、txt、dat、c、
cpp、obj、exe、ppt、
bmp等)

10.1.2 文件名

- 文件路径表示文件在存储设备中的位置。如：

文件名

D: \CC\temp\file1.dat

- 表示file1.dat文件存放在D盘中的CC目录下的temp子目录下面

10.1.3 文件的分类

- 根据数据的组织形式，数据文件可分为**ASCII文件**和**二进制文件**。
 - 数据在内存中是以二进制形式存储的，如果不加转换地输出到外存，就是**二进制文件**（也称**映像文件**）
 - 如果要求在外存上以**ASCII**代码形式存储，则需要在存储前进行转换，即**ASCII文件**又称**文本文件**
 - 每一个字节放一个字符的**ASCII**代码

10.1.3 文件的分类

- 字符一律以ASCII形式存储
- 数值型数据既可以用ASCII形式存储，也可以用二进制形式存储
 - 如整数10000，如果用ASCII码形式输出到磁盘，则在磁盘中占5个字节(每一个字符占一个字节)，而用二进制形式输出，则在磁盘上只占4个字节(用VC++ C时)

ASCII形式

00110001	00110000	00110000	00110000	00110000
----------	----------	----------	----------	----------

(1) (0) (0) (0) (0)

二进制形式

00000000	00000000	00100111	00010000
----------	----------	----------	----------

(10000)

ASCII 码文件

```

hello.c - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

/* File : hello.c
 * -----
 * ...
 */

#include <stdio.h>

main ()
{
    printf ("Hello, world.\n");
}

```

文本模式

```

/* File : hello.
c.. * -----
-----.. * .....
*/.....#include
<stdio.h>...mai
n ()..{..    prin
tf ("Hello, worl
d.\n");..}..

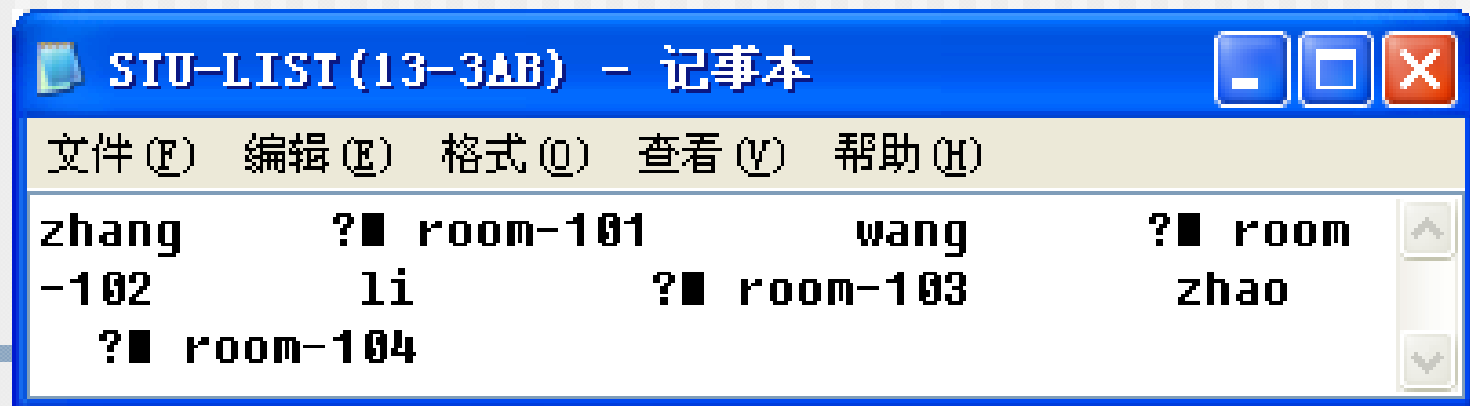
```

十六进制模式

```

2F 2A 20 46 69 6C 65 20 3A 20 68 65 6C 6C 6F 2E
63 0D 0A 20 2A 20 2D 2D 2D 2D 2D 2D 2D 2D 2D 2D
2D 2D 2D 2D 2D 2D 0D 0A 20 2A 20 2E 2E 2E 0D 0A
20 2A 2F 0D 0A 0D 0A 23 69 6E 63 6C 75 64 65 20
3C 73 74 64 69 6F 2E 68 3E 0D 0A 0D 0A 6D 61 69
6E 20 28 29 0D 0A 7B 0D 0A 20 20 20 70 72 69 6E
74 66 20 28 22 48 65 6C 6C 6F 2C 20 77 6F 72 6C
64 2E 5C 6E 22 29 3E 0D 0A 7D 0D 0A

```



十六进制模式

```

7A 68 61 6E 67 00 00 00 00 00 E9 03 13 00 72 6F
6F 6D 2D 31 30 31 00 00 00 00 00 00 00 77 61 6E
67 00 00 00 00 00 00 00 EA 03 14 00 72 6F 6F 6D 2D
31 30 32 00 00 00 00 00 00 00 6C 69 00 00 00 00
00 00 00 00 EB 03 14 00 72 6F 6F 6D 2D 31 30 33
00 00 00 00 00 00 00 7A 68 61 6F 00 00 00 00 00
00 EC 03 16 00 72 6F 6F 6D 2D 31 30 34 00 00 00
00 00 00 00

```

```

zhang.....?..ro
om-101.....wan
g.....?..room-
102.....li....
....?..room-103
.....zhao.....
.?..room-104...
.....

```

```

struct student_type
{
    char name[10];
    int num;
    int age;
    char addr[15];
}stud[4];

```

```

zhang 1001 19 room_101
wang 1002 20 room_102
li 1003 20 room_103
zhao 1004 22 room_104

```

10.1.3 文件的分类

- ASCII文件和二进制文件的比较：
 - 用**ASCII**码形式输出与字符一一对应，一个字节代表一个字符，便于对字符进行逐个处理，也便于输出，但占存储空间较多，而且要花费转换时间。
 - 用二进制形式输出数值，可以节省外存空间和转换时间，但不能直接输出字符形式。
 - 常用于保存中间结果。

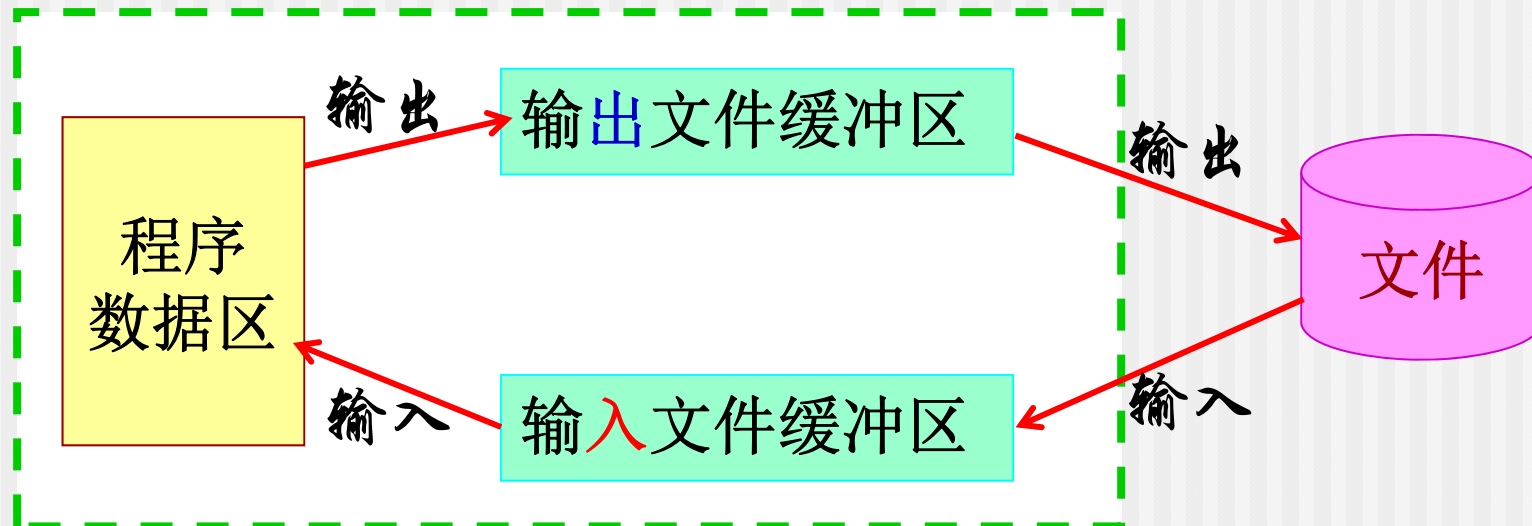
10.1.3 文件的分类

- 输入文本文件时，系统自动将回车换行符（**ASCII**码值分别为**10**和**13**）转换为一个换行符；向文本文件输出时，则将换行符转换为回车和换行两个字符；因此，读/写的字符数量可能与文本文件中存储的字符数量不一致。二进制文件无此转换。
- **C文件**是一个字节流或二进制流。——**流式文件**
 - 把数据看作是一连串的字符（字节），而不考虑记录的界限。也就是说，**C文件**不是由记录(**record**)组成的（不同于**pascal**等其它高级语言），其存取的基本单位是字符（字节）。
 - 不考虑文件的内容，文件的内容如何解释完全依赖于相应的存取程序。（不要求以回车换行符作为数据/记录之间的间隔。）

10.1.4 文件缓冲区

■ “缓冲文件系统”和“非缓冲文件系统”

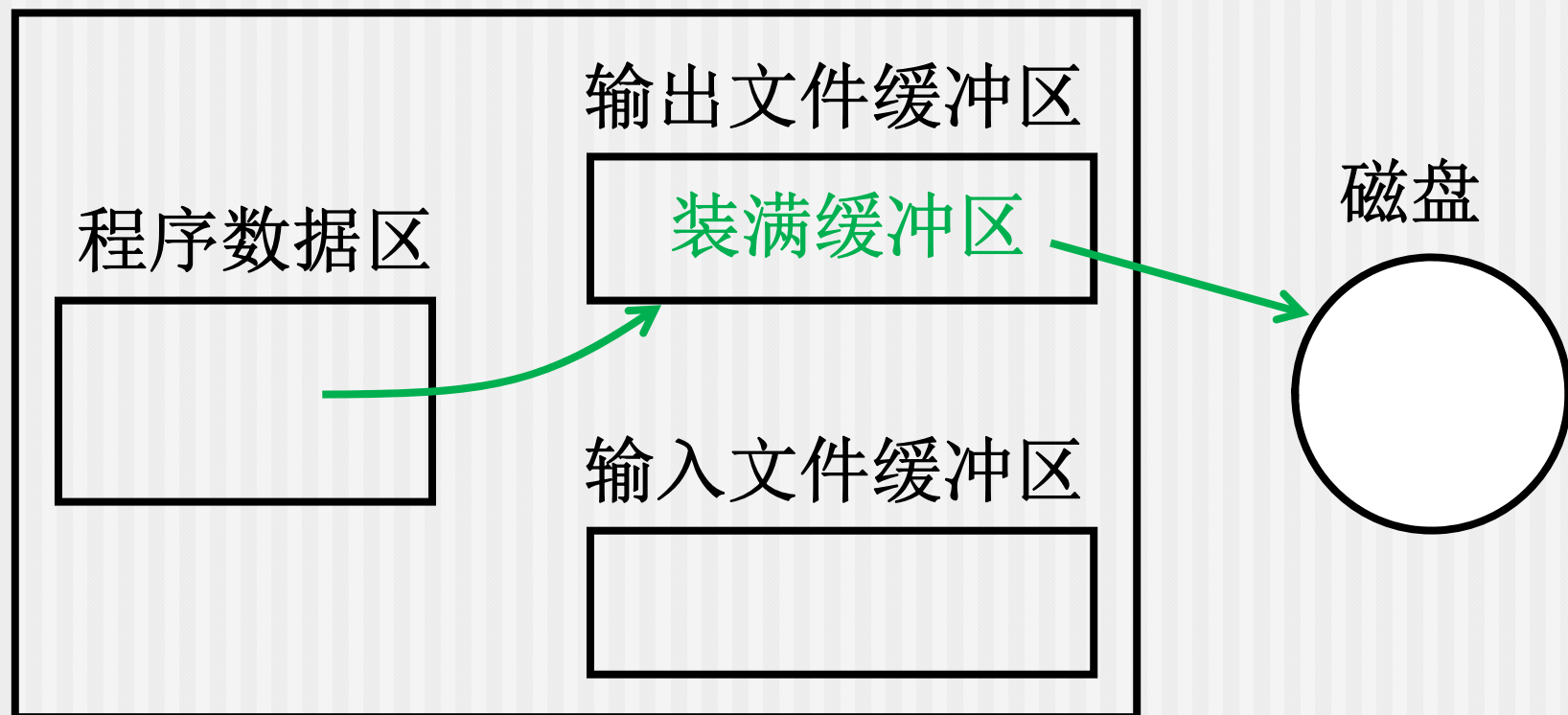
- 缓冲文件系统：系统自动地在内存区中为每一个正在使用的文件开辟一个缓冲区。内存和磁盘的数据交换必须通过该缓冲区（大小一般为512字节）。
- 非缓冲文件系统：系统不自动开辟确定大小的缓冲区，而由程序为每个文件设定缓冲区。（如UNIX）
 - 每次读写都需启动磁盘操作，将大大降低传输效率(磁盘是慢速设备)，且降低磁盘的使用寿命
- ANSI C标准采用缓冲文件系统。



10.1.4 文件缓冲区

- ANSI C标准采用“缓冲文件系统”处理数据文件
 - 从内存向磁盘输出数据

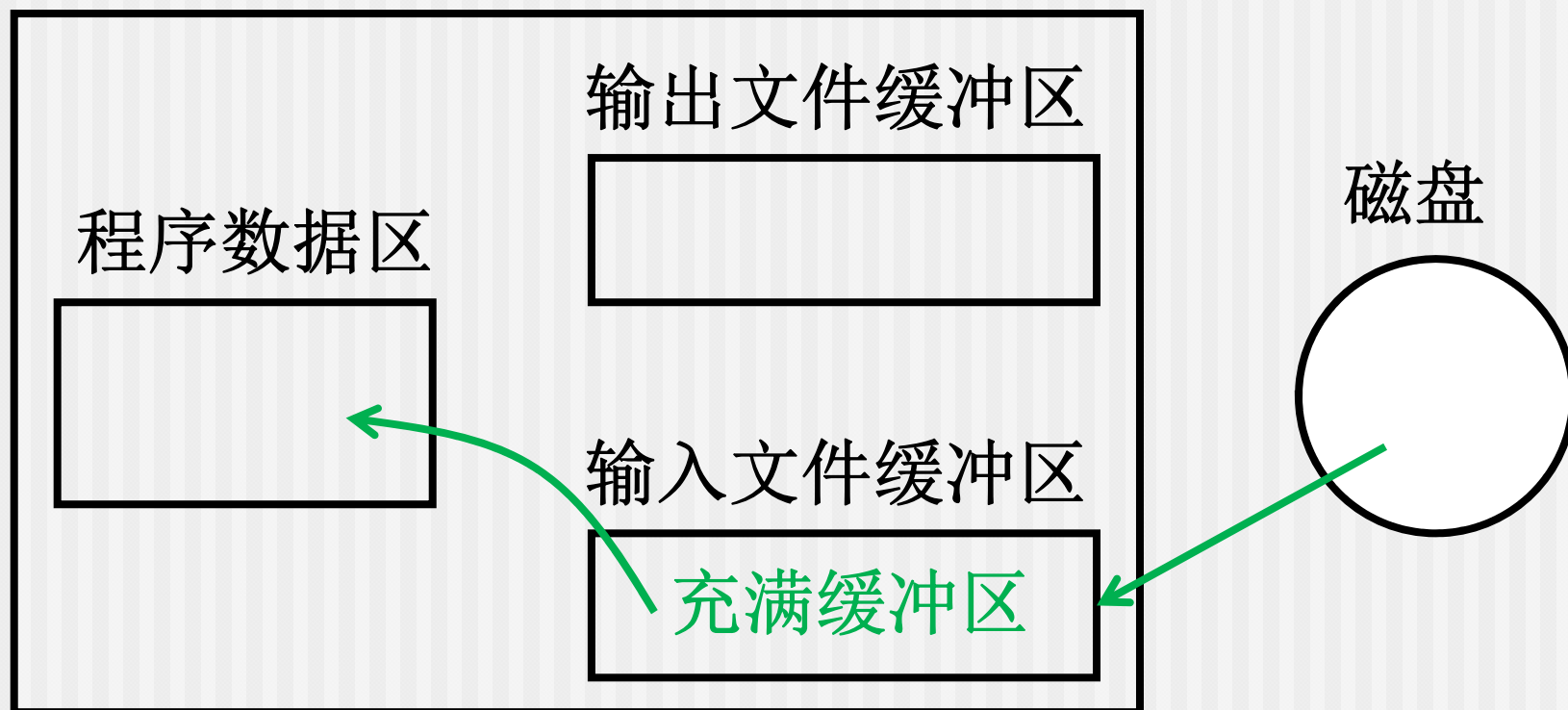
- 必须先送到内存中的缓冲区，装满缓冲区后才一起送到磁盘去



10.1.4 文件缓冲区

- ANSI C标准采用“缓冲文件系统”处理数据文件
 - 从磁盘向计算机读入数据

- 一次从磁盘文件将一批数据输入到内存缓冲区（充满缓冲区），然后再从缓冲区逐个地将数据送到程序数据区（给程序变量）



10.1.5 文件类型指针

- 缓冲文件系统中，关键的概念是“**文件类型指针**”，简称“**文件指针**”
 - 每个被使用的文件都在内存中开辟一个相应的文件信息区，用来存放文件的有关信息（如文件的名称、文件状态及文件当前位置等）
 - 这些信息是保存在一个结构体变量中的。该结构体类型是由系统声明的，取名为**FILE**
- 声明**FILE**结构体类型的信息包含在头文件“**stdio.h**”中
- 一般设置一个指向**FILE**类型变量的指针变量，然后通过它来引用这些**FILE**类型变量

10.1.5 文件类型指针

■ 形如:

```
typedef struct{
    short          level; /* 缓冲区“满”或“空”的程度 */
    unsigned       flags; /* 文件状态标志 */
    char           fd;     /* 文件描述符 */
    unsigned char hold; /* 如缓冲区无内容不读取字符 */
    short          bsize; /* 缓冲区大小 */
    unsigned char * buffer; /* 数据缓冲区的位置 */
    unsigned char * curp; /* 位置指针, 当前的指向 */
    unsigned       istemp; /* 临时文件指示器 */
    short          token; /* 用于有效性检查 */
} FILE;
```

- 不同的C语言版本, 定义可能略有不同。

10.1.5 文件类型指针

25

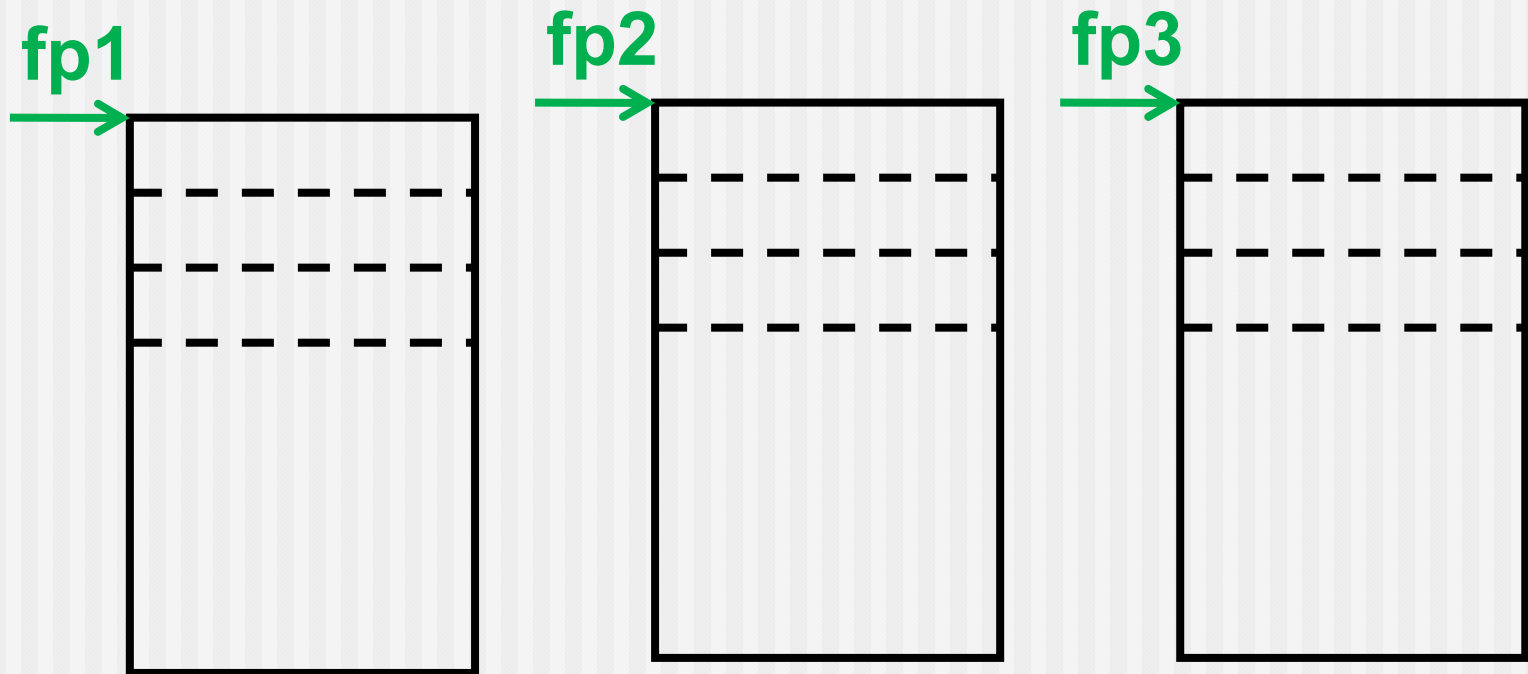
- 可用FILE类型定义若干个FILE类型的变量，以便存放若干个文件的信息。
 - 通过这些变量中的信息就能访问相应文件的信息区；
 - 这些信息在打开文件时由系统自动放入；
 - 通常，不通过变量名来引用这些变量。
- 多数的文件处理函数要求传递的参数和返回值是FILE类型的结构体指针，更常使用如下的定义：

FILE * <文件指针变量名表>;

- **文件指针**指向某一文件的文件信息区（确切地说是指向内存中存放该文件信息的结构体变量），通过该信息区中的信息就可以访问该文件，即通过文件指针变量能够找到与之关联的文件。
- 若需要同时打开n个文件，就需要定义n个文件指针变量，分别指向这n个文件

10.1.5 文件类型指针

```
FILE *fp1, *fp2, *fp3;
```



文件**f1**的
文件信息区

文件**f2**的
文件信息区

文件**f3**的
文件信息区

10.2 打开与关闭文件

- “打开”打开一个文件时，需要通知编译器的三个信息：
 - ① 需要打开(即准备访问)的文件名；
 - ② 使用文件的方式(“读”还是“写”等)；
 - ③ 让哪一个文件指针变量指向被打开的文件。

#include <stdio.h>

FILE *fopen (const char *filename , const char *mode);

成功时返回被打开文件的文件指针，失败时返回**NULL**。

- 常用形式：

```
FILE *fp;
```

```
...
```

```
if ((fp=fopen("file1", "r"))==NULL) {
```

```
    printf("cannot open this file.\n");
```

```
    exit(0); // 关闭所有文件，终止正在执行的程序
```

```
}
```

```
...
```

10.2 打开与关闭文件

■ **mode**的取值：（P335 表10-1）

【文本文件】

- “r”——只读，文件必须已经存在
- “w”——只写，新建文件（文件若已存在则先删除）
- “a”——在文件尾追加，只写，文件必须已存在
- “r+”、“w+”——可读可写（文件要求分别同上）
- “a+”——可追加可读，文件必须已存在

【二进制文件】

- “rb”、“wb”、“ab”、“rb+”、“wb+”、“ab+”

■ 不同C版本对**mode**的定义有所不同！

■ **C**的三个标准文件：标准输入（**stdin**）、标准输出（**stdout**）、标准出错输出（**stderr**）。

10.2 打开与关闭文件

- “关闭”文件（**fclose**函数）——撤销文件信息区和文件缓冲区，使文件指针变量不再指向该文件，即与文件“脱钩”。

```
#include <stdio.h>
```

```
int fclose(FILE *fp);
```

成功关闭文件返回0，失败时返回**EOF**(即-1)。

- **fclose**会先把缓冲区中的数据输出到磁盘文件中，再撤销文件信息区。
- 不关闭文件将可能导致数据丢失，因为数据可能还在缓冲区中，未真正写到外设文件中。
- 关闭文件后，文件指针变量只是不再指向该文件，而不是释放了该变量，它仍可用于指向其它文件。

10.3 顺序读写数据文件

即对文件读写数据的顺序和数据在文件中的物理顺序一致。

■ 读/写一个字符

■ fputc函数（putc函数）

#include <stdio.h>

int fputc(int c, FILE *fp);

把字符**c**写到文件指针变量**fp**所指向的文件中，
成功则返回**c**的值，失败则返回**EOF**(即-1)

- **fp**应为指向已打开文件的指针。

■ putc(c,fp)是宏，功能相当于fputc

#define putc(c,fp) fputc(c,fp)

10.3 顺序读写数据文件

即对文件读写数据的顺序和数据在文件中的物理顺序一致。

■ 读/写一个字符

■ fgetc函数（getc函数）

#include <stdio.h>

int fgetc(FILE *fp);

从fp指向的文件读入一个字符，成功则返回所读取的那个字符的ASCII码值，失败则返回EOF(即-1)

- fp应为指向已打开文件的指针。

■ getc(fp)是宏，功能相当于fgetc

#define getc(fp) fgetc(fp)

10.3 顺序读写数据文件

■ 常见的读取字符操作

- 从一个**文本文件**顺序读入字符并在屏幕上显示：

```
ch = fgetc (fp) ;  
while (ch != EOF)  
{  
    putchar (ch) ;  
    ch = fgetc (fp) ;  
}
```

【注意】

- EOF（即-1）不是可输出字符，因此不能在屏幕上显示。
- 由于字符的ASCII码不可能出现-1，因此当读入的字符值等于EOF时，表示读入的已不是正常的字符而是文件结束符。

10.3 顺序读写数据文件

■ 常见的读取字符操作

- 从一个**二进制文件**顺序读入字符（字节）：

```
while ( ! feof ( fp ) )  
{  
    ch = fgetc ( fp ) ;  
    .....  
}
```

【注意】

- ANSI C提供一个**feof ()** 函数来判断文件是否真的结束。如果是文件结束，函数**feof (fp)** 的值为 1（真）；否则为 0（假）。以上也适用于文本文件的读取。

运行情况如下：

f i l e 1 . c (输入磁盘文件名)

c o m p u t e r a n d c # (输入一个字符串)

c o m p u t e r a n d c (输出一个字符串)

```
#include <stdio.h>
```

```
int main(void)
```

```
{ FILE *fp; char ch,filename[10];
```

```
scanf("%s",filename);
```

```
if((fp=fopen(filename,"w"))==NULL) {
```

```
printf("cannot open file\n");
```

```
exit(0); /*终止程序*/}
```

```
ch=getchar( ); /*接收执行scanf时最后输入的回车符 */
```

```
ch=getchar( ); /* 接收输入的第一个字符 */
```

```
while (ch!='#') {
```

```
    fputc(ch,fp); putchar(ch);
```

```
    ch=getchar(); }
```

```
fclose(fp); putchar(10); return 0;
```

```
}
```


【例10.2】 将一个磁盘文件中的信息复制到另一个磁盘³⁵文件中。

【BUG】 file2.c的最后会多出一个ASCII值为FF的字符

【原因】 用fgetc读取文件的最后一个字符后，文件未变成eof状态，要等到再次读取的时候，fgetc才返回EOF，并且文件状态才是eof的，而此时，那个EOF已经被put到输出的文件中了。

程序运行结果是将 f i l e 1 . c 文件中的内容复制到 f i l e 2 . c 中去。

```
if ((out=fopen(outfile, "w")) == NULL) //或 WB
{ printf("cannot open outfile\n"); exit(0); }
while (!feof(in))
{ ch=fgetc(in);
  if (ch!=EOF) { fputc(ch,out); putchar(ch); }
}
fclose(in); fclose(out);
return 0;
}
```

【例13.2'】 将一个磁盘文件中的信息复制到另一个磁盘文件中。³⁶

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[ ])
{ FILE *in, *out; char ch;
  if (argc != 3) {
    printf("You forgot to enter a filename\n");
    exit(0); } }
```

运行方法:

设经编译连接后得到的可执行文件名为a.exe, 则在DOS命令工作模式下, 可以输入以下的命令行:

C>a f i l e 1 . c f i l e 2 . c
f i l e 1 . c 和 f i l e 2 . c, 分别输入到 a r g v [1]
和 a r g v [2] 中, a r g v [0] 的内容为a, a r g c 的
值等于 3 。

10.3 顺序读写数据文件

■ 读/写一个字符串

■ fgets函数

#include <stdio.h>

char *fgets(char *str, int n, FILE *fp);

从fp指向的文件输入一个长度为n-1的字符串，并在最后加一个'\0'，将这n个字符存放到字符数组str中；成功返回str的首地址，失败则返回EOF(即-1)

- n为要求读入的字符个数+1；
- 若在读完n-1个字符之前遇到换行符'\n'或文件结束符EOF，则读入结束，但'\n'也将作为一个字符读入。

10.3 顺序读写数据文件

■ 读/写一个字符串

■ fputs函数

#include <stdio.h>

int fputs(char *str, FILE *fp);

将str所指向的字符串输出到fp所指向的文件中，成功返回0，失败则返回EOF。

- Str可以是字符串常量、字符数组名或者字符型指针；
- 字符串结尾处的'\0'不输出。

【例10.3】从键盘读入若干个字符串，对它们按字母大小的顺序排序，然后把排好序的字符串送到磁盘文件中保存。³⁹

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main( )
{ FILE *fp; char str[3][10],temp[10]; int i,j,k,n=3;
  printf("Enter strings:\n");
  for (i=0; i<n; i++) gets(str[i]);
  for (i=0; i<n-1; i++)
  { k=i;
    for (j=i+1; j<n; j++)
      if (strcmp(str[k],str[j])>0) k=j;
    if (k!=i)
    { strcpy(temp, str[i]);
      strcpy(str[i],str[k]);
      strcpy(str[k],temp); }
  }
```



```
if ((fp=fopen("D:\\CC\\string.dat","w"))  
    ==NULL)  
{ printf("can't open file!\n"); exit(0);}  
printf("\nThe new sequence:\n");  
for (i=0; i<n; i++)  
{ fputs(str[i],fp); fputs("\n",fp);  
  printf("%s\n",str[i]);  
}  
fclose(fp);  
return 0;  
}
```

如果无此语句，前后两次输出的字符串之间无分隔，将连成一片。

编写程序从文件string.dat中读回字符串，并在屏幕上⁴¹显示。

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main( )
{ FILE *fp; char str[3][10]; int i=0;
  if ((fp=fopen("D:\\CC\\string.dat","r"))
      ==NULL)
  { printf("can't open file! \n");
    exit(0);
  }
  while (fgets(str[i],10,fp)!=NULL)
  { printf("%s",str[i]);
    i++;
  }
fclose(fp);
  return 0;
}
```

13.4 文件的读写（6）

■ 格式化方式的读/写数据（`fscanf()`和`fprintf()`）

```
#include <stdio.h>
```

```
int fscanf(FILE *fp, const char *format, ...);
```

```
int fprintf(FILE *fp, const char *format, ...);
```

返回成功输入/输出的数据项的个数。

- 特点：使用使用方便，容易理解，但由于在输入时要将ASCII码转换为二进制形式，在输出时又要将二进制形式转换成字符，花费时间比较多。
- 在内存与磁盘频繁交换数据的情况下，最好不用`fprintf`和`fscanf`函数，而用`fread`和`fwrite`函数。

```
fprintf(fp, "%d, %6.2f", i, f);
```

```
fscanf(fp, "%d, %f", &i, &f);
```

13.4 文件的读写（5）

■ 二进制方式读/写数据块（`fread()`和`fwrite()`）

```
#include <stdio.h>
```

```
size_t fread(void *buffer, size_t size, size_t n, FILE *fp);
```

```
size_t fwrite(void *buffer, size_t size, size_t n, FILE *fp);
```

返回实际输入/输出的数据项的个数。

【参数说明】

- **buffer**: 一个指针。（均指起始地址）
 - 对**fread**来说，用来存放从文件读入的数据的存储区的地址。
 - 对**fwrite**来说，把此地址开始的存储区中的数据向文件输出。
- **size**: 要读写的字节数。
- **n**: 要读写多少个数据项（每个长度为**size**个字节）。
- **fp**: 文件型指针。

【例】从键盘输入 4 个学生的有关数据，然后把它们转存到磁盘文件上去。

■ 结构类型

```
#define SIZE 4
struct student_type
{ char name[10];
  int num;
  int age;
  char addr[15];
}stud[SIZE]; /*定义结构*/
```

■ 输入/输出操作

```
for ( i = 0 ; i < SIZE ; i ++ )
    fread(&stud [i] , sizeof(struct student_type), 1, fp);
for ( i = 0 ; i < SIZE ; i ++ )
    fwrite(&stud [i] , sizeof(struct student_type), 1, fp);
```


【例10.4】从键盘输入10个学生的有关数据，然后把⁴⁵它们转存到磁盘文件上去。

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 10
struct student_type
{ char name[10];
  int num;
  int age;
  char addr[15];
} stud[SIZE];

void save()
{ FILE *fp; int i;
  if ((fp=fopen("stu.dat","wb"))==NULL)
  { printf("cannot open file\n"); return; }
  for (i=0; i<SIZE; i++)
    if (fwrite(&stud[i], sizeof(struct student_type),1,fp)!=1)
      printf("file write error\n");
  fclose(fp);
}
```

```
int main( )
{ int i;
  printf("Please enter data of students:\n");
  for (i=0; i<SIZE; i++)
    scanf("%s%d%d%s",stud[i].name, &stud[i].num,
          &stud[i].age, stud[i].addr);
  save();
  return 0;
}
```

可以用以下程序从磁盘文件“stu.dat”中读出数据，并⁴⁷
在屏幕上输出。

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 10
struct student_type
{ char name[10];
  int num;
  int age;
  char addr[15];
}stud[SIZE],

int main( )
{ int i; FILE *fp;
  if ((fp=fopen("stu.dat","rb"))==NULL)
  { printf("cannot open file\n"); exit(0); }
  for (i=0; i<SIZE; i++) while (!feof(fp))
  {if (fread(&stud[i], sizeof(struct student_type),1,fp))!=1) break;
    printf("%-10s %4d %4d %-15s\n", stud[i].name,
        stud[i].num,stud[i].age, stud[i].addr); }
  fclose(fp);
  return 0;
}
```

【例10.4'】从键盘输入10个学生的有关数据，然后把⁴⁸它们转存到磁盘文件上去。

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 10
struct student_type
{ char name[10];
  int num;
  int age;
  char addr[15];
}stud[SIZE];

void save()
{ FILE *fp; int i;
  if ((fp=fopen("stu.dat","wb"))==NULL)
  { printf("cannot open file\n"); return; }
  for (i=0; i<SIZE; i++)
    if (fwrite(&stud[i], sizeof(struct student_type),1,fp)!=1)
      printf("file write error\n");
  fclose(fp);
}
```

void load() //从磁盘文件中读取二进制数据，存放to stud数组中

```
{ FILE *fp; int i;
  if ((fp=fopen("stu.dat","rb")) == NULL)
  { printf("cannot open file\n"); return; }
  for (i=0; i<SIZE; i++)
    if (fread(&stud[i], sizeof(struct student_type), 1,fp)!=1)
    { if (feof(fp))
      { fclose(fp); return; }
      printf("file read error\n");
    }
  fclose(fp);
}
```

int main()

```
{ int i;
  printf("Please enter data of students:\n");
  for (i=0; i<SIZE; i++)
    scanf("%s%d%d%s",stud[i].name, &stud[i].num,
        &stud[i].age, stud[i].addr);
```

save();

load();

return 0;

}

■ 其他读写函数

- 读写一个字（整数）putw()和getw()

```
#include <stdio.h>
```

```
int putw(int i,FILE * fp);
```

```
int getw(FILE * fp);
```

- 读写其他类型数据——用户自定义

如: putfloat(float num,FILE *fp)

```
{ char s; int count;
```

```
  s = (char*)&num;
```

```
  for(count = 0;count < 4;count++)
```

```
    putc(s[count],fp);
```

```
}
```

10.4 随机读写数据文件

- **顺序读写**：按数据在文件中的物理位置次序进行读写；
 - 容易理解，容易操作，但效率不高；
- **随机读写**：按任意顺序对任何位置上的数据进行读写。

10.4.1 文件位置标记及其定位

- 系统为每个文件设置了一个**文件读写位置指针**（简称**文件位置标记**或者**文件标记**），用来指示“接下来要读写的下一个字符的位置”。
- 若顺序读写一个文件，且每次读写一个字符，则读写完一个字符后，该文件的位置标记就自动移动到下一个字符的位置。



10.4 随机读写数据文件

- **顺序读写**：按数据在文件中的物理位置次序进行读写；
 - 容易理解，容易操作，但效率不高；
- **随机读写**：按任意顺序对任何位置上的数据进行读写。

10.4.1 文件位置标记及其定位

- 随机读写，即根据操作需要，人为地移动文件位置标记的位置，可以向前移、向后移、移到文件头或文件尾
- 相关函数有：
 - rewind函数
 - fseek函数
 - ftell函数

- 使文件位置标记重新返回文件的开头

#include <stdio.h>

void rewind(FILE *fp);

【例10.5】将一个磁盘文件先显示到屏幕上，然后复制到另一个文件中。

```
#include <stdio.h>
```

```
int main()
```

```
{ FILE *fp1, *fp2;
```

```
  fp1 = fopen("file1.dat", "r");
```

```
  fp2 = fopen("file2.dat", "w");
```

```
  while(!feof(fp1)) putchar(getc(fp1)); //输出到屏幕
```

```
  putchar(10);
```

```
  rewind(fp1);
```

```
  while(!feof(fp1)) putc(getc(fp1), fp2); //输出到文件
```

```
  fclose(fp1); fclose(fp2);
```

```
  return 0;
```

```
}
```

有多处**BUG!**

■ 改变文件位置标记

```
#include <stdio.h>
```

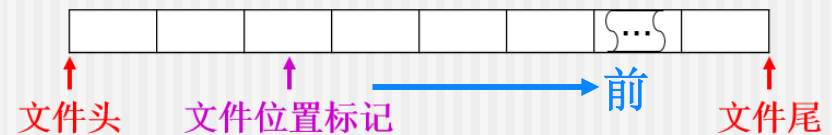
```
int fseek(FILE *fp, long offset, int whence);
```

■ whence: 起始点

文件开头	SEEK_SET	0
文件当前位置	SEEK_CUR	1
文件末尾	SEEK_END	2

■ offset: 位移量, 以起始点为基点, 向前移动的字节数; 一般要求为long型。

例如:



■ `fseek(fp, 100L, 0);`

将文件位置标记向前移到离文件头**100**个字节处

■ `fseek(fp, 50L, 1);`

将文件位置标记向前移到离当前位置**50**个字节处

■ `fseek(fp, -10L, 2);`

将文件位置标记从文件末尾处向后退 **10** 个字节

■ `fseek`一般用于二进制文件, 因为文本文件要发生字符转换, 计算位置时往往发生混乱。

■ 测定文件位置标记的当前位置

```
#include <stdio.h>
```

```
long ftell(FILE *fp);
```

返回流式文件`fp`中文件位置标记的当前位置，用相对于文件开头的位移量来表示；出错时返回-1L。

例如：

```
i = ftell(fp);
```

```
if (i == -1L) printf("error\n");
```

【例10.6】在磁盘文件上存有10个学生的数据。要求将第1、⁵⁷
3、5、7、9个学生数据读取出来并显示在屏幕上。

```
#include <stdlib.h>
#include <stdio.h>
struct student_type
{ char name[10]; int num; int age; char addr[15];
} stud[10];

int main()
{ int i; FILE *fp;
  if ((fp=fopen("stud-dat","rb"))==NULL)
  { printf("can not open file\n"); exit(0); }
  for (i=0; i<10; i+=2)
  { fseek(fp,i*sizeof(struct student_type),0);
    fread(&stud[i], sizeof(struct student_type),1,fp);
    printf("%-10s %4d %4d %-15s\n",stud[i].name,
          stud[i].num, stud[i].age, stud[i].addr); }
  fclose(fp);
  return 0;
}
```

10.5 文件读写的出错检测

58

- 检测各种输入输出函数调用是否出错

```
#include <stdio.h>
```

```
int ferror(FILE *stream);
```

返回0，表示未出错；返回非0，表示出错。

- 对同一个文件每一次调用输入输出函数，均产生一个新的ferror函数值。
- 应当在调用一个输入输出函数后立即检查ferror函数的值，否则信息会丢失。
- 在执行fopen函数时，ferror函数的初始值自动置为0。

■ 出错标志置0

```
#include <stdio.h>
```

```
void clearerr(FILE *fp);
```

使文件错误标志和文件结束标志置为0。

- 只要出现错误标志，就一直保留，直到对同一文件调用clearerr函数或rewind函数，或任何其他一个输入输出函数。